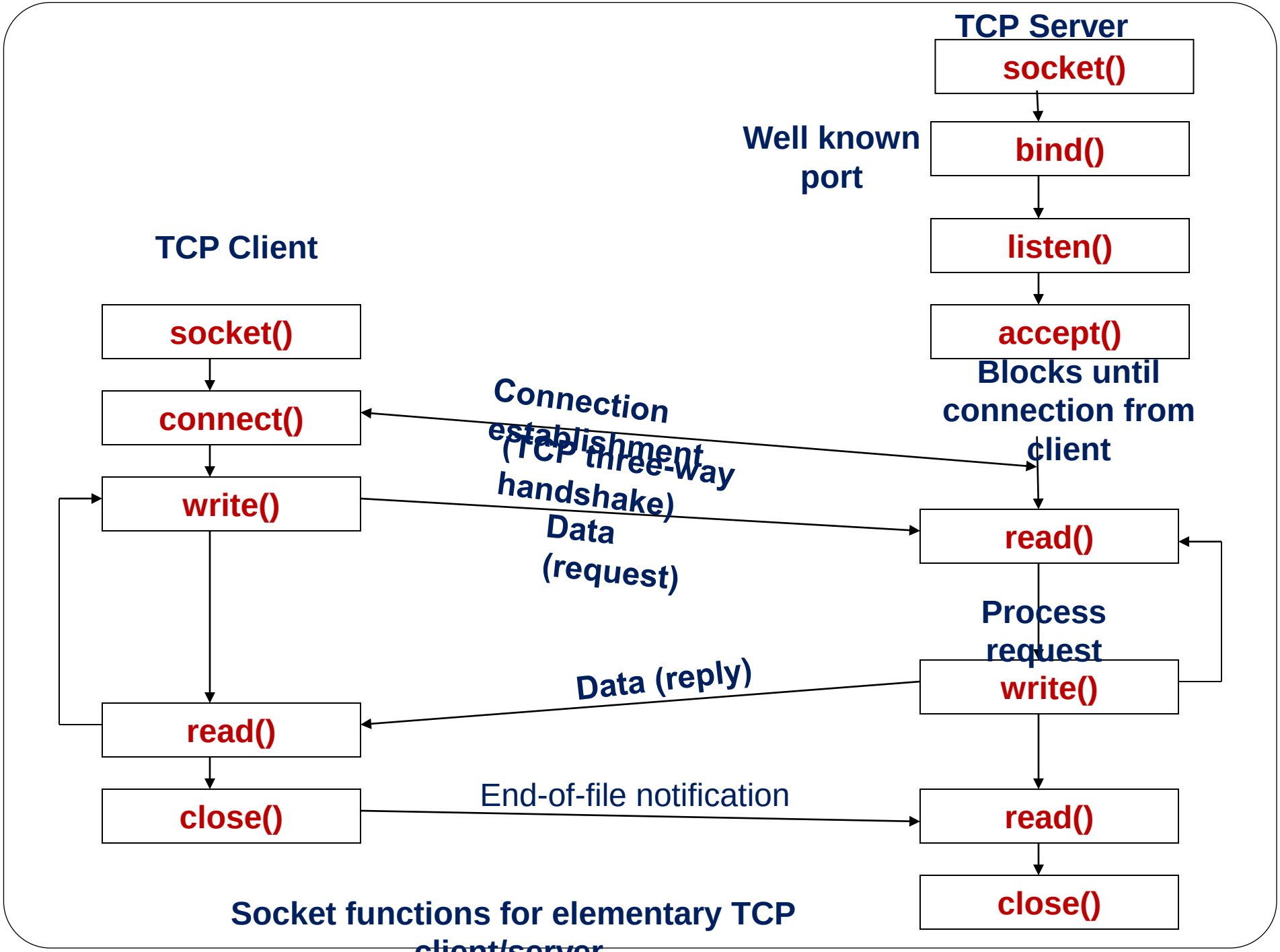
Elementary TCP Sockets

Introduction

- Time line of typical Client Server
- Socket functions
- Concurrent servers



Socket functions for elementary TCP client/server.

Socket Function

- To perform network I/O, the first thing a process must do is to call the socket function, specifying the type of communication protocol desired.

  #include <sys/socket.h>

  int socket (int family, int type, int protocol);

  Returns: non-negative descriptor if OK, -1 on error

- family specifies the protocol family.
- type specifies the socket type
- protocol argument should be set to the specific protocol type

Protocol family constants

Family	Description
AF_INET	IPv4 protocol
AF_INET6	IPv6 protocol
AF_LOCAL	UNIX DOMAIN PROTOCOL
AF_ROUTE	Routing socket
AF_KEY	Key socket

Type of Socket

Type	Description
SOCK_STREAM	Stream socket
SOCK_DGRAM	Datagram socket
SOCK_SEQPACKET	Sequenced packet socket
SOCK_RAW	Raw socket

Protocol of Sockets for AF_INET or AF_INET6

protocol	description
IPPROTO_TCP	TCP transport protocol
IPPROTO_UDP	UDP transport protocol
IPPROTO_SCTP	SCTP transport protocol

Combinations of family and type for the socket function

	AF_INET	AF_INET6	AF_LOCAL	AF_ROUTE	AF_KEY
SOCK_STREAM	TCP|SCTP	TCP|SCTP	Yes		
SOCK_DGRAM	UDP	UDP	Yes		
SOCK_SEQPACKET	SCTP	SCTP	Yes		
SOCK_RAW	IPv4	IPv6		Yes	Yes

connect function

- Used by a TCP client to establish a connection with a TCP server.

```
#include <sys/socket.h>
int connect(int sockfd, const struct sockaddr
  *servaddr, socklen_t addrlen);
Returns: 0 if OK, -1 on error
```

  - Returns : 0 if successful connect, -1 otherwise
  - Sockfd – socket descriptor returned by the socket function
  - second & third arguments are pointers to a socket address structure and its size.
  - socket address structure must contain the IP address and port number of the server.
  - Client no need to call bind() before the connect function.

connect function

- In case of TCP socket , connect function initiates TCP’s three-way handshake.
- Connect function returns only when the connection is established or an error occurs.

Error returns by connect

- possible error returns of connect function
  1. ETIMEDOUT : no response from server
  2. RST : server process is not running
  3. EHOSTUNREACH : client's SYN unreachable from some intermediate router.

Error returns by connect

1. ETIMEDOUT- if the client TCP receives no response to its SYN segment, ETIMEDOUT is returned.
2. RST- if the servers response to the client's SYN is reset(RST), this indicates that no process is waiting for connections on the server host at the port specified.

   called hard error

   error ECONNREFUSED is returned to the client as soon as the RST is received.

Error returns by connect

RST is a type of TCP segment that is sent by TCP when something is wrong.

Three conditions that generate an RST are:

i. when a SYN arrives for a port that has no listening server

ii. TCP wants to abort an existing TCP connection

iii. When TCP receives a segment for a connection that does not exist.

Error returns by connect

- EHOSTUNREACH- if the client's SYN elicits an ICMP "destination unreachable" from some intermediate router, this is considered a soft error.

  client's TCP keeps sending SYN's

  if no response is received after some fixed amount of time (say a 75s for 4.3 BSD)

  EHOSTUNREACH or ENETUNREACH is returned to the process.

  It is also possible that

  -remote system is not reachable by any route, or

  -connect call returns without waiting at all.

bind function

- Assigns a local protocol address to a socket.
- With the Internet protocols, the protocol address is the combination of either a 32-bit IPv4 address or 128-bit IPv6 address, along with a 16-bit TCP or UDP port number.

```
#include <sys/socket.h>
int bind (int sockfd, const struct sockaddr
   *myaddr, socklen_t addrlen);
```

Returns: 0 if OK,-1 on error

- Second argument is a pointer to a protocol-specific address,
- Third argument is the size of the address structure.

bind function

- With TCP, calling bind lets us specify a port number, an IP address, both, or neither.
- Servers bind their well-known port when they start.
- kernel chooses an ephemeral port for the socket when either connect or listen is called.
- A process can bind specific IP address to its socket.
- Normally, a TCP client does not bind an IP address to its socket. The kernel chooses the source IP address when the socket is connected, based on the outgoing interface that is used, which in turn is based on the route required to reach the server.
- If a TCP server does not bind an IP address to its socket, the kernel uses the destination IP address of the client's SYN as the server's source IP address .

Table summarizes the values to which we set sin_addr and sin_port, or sin6_addr and sin6_port, depending on the desired result.

Process specifies		Result
IP address	port	
Wildcard	0	kernel chooses IP address and port
Wildcard	Nonzero	kernel chooses IP address, process specifies port
Local IP address	0	Process specifies IP address, kernel chooses port
Local IP address	nonzero	process specifies IP address **and port**

Bind function

- If port number of 0, the kernel chooses an ephemeral port when bind is called.
- If port assigned a wildcard IP address, the kernel does not choose the local IP address until either the socket is connected (TCP) or a datagram is sent on the socket (UDP).
- With IPv4, the wildcard address is specified by the constant INADDR_ANY, whose value is normally 0.
- Server calls getsockname to obtain the destination IP address from the client, then handles the client request based on the IP address to which the connection was issued.

listen function

#include <sys/socket.h>

#int listen (int sockfd, int backlog);

Returns: 0 if OK, -1 on error

Called only by a TCP server and it performs two actions.

1. When a socket is created by the socket function- active socket.

   Converts an unconnected socket into a passive socket, indicating that the kernel should accept incoming connection requests directed to this socket. Call to listen moves the socket from CLOSED state to the LISTEN state.

2. backlog – specifies the maximum number of connections the kernel should queue for this socket.

listen function

- normally called after both the *socket* and *bind* and must be called before the *accept* function.
- kernel maintains two queues- *incomplete connection queue*

and *completed connection queue.*

Incomplete connection queue- contains an entry for each SYN that has arrived from a client for which the server is awaiting completion of the TCP three-way handshake.

Completed connection queue-contains an entry for each client with whom the TCP three-way handshake has completed.

Figure 4.6 depicts these two queues for a given listening socket.

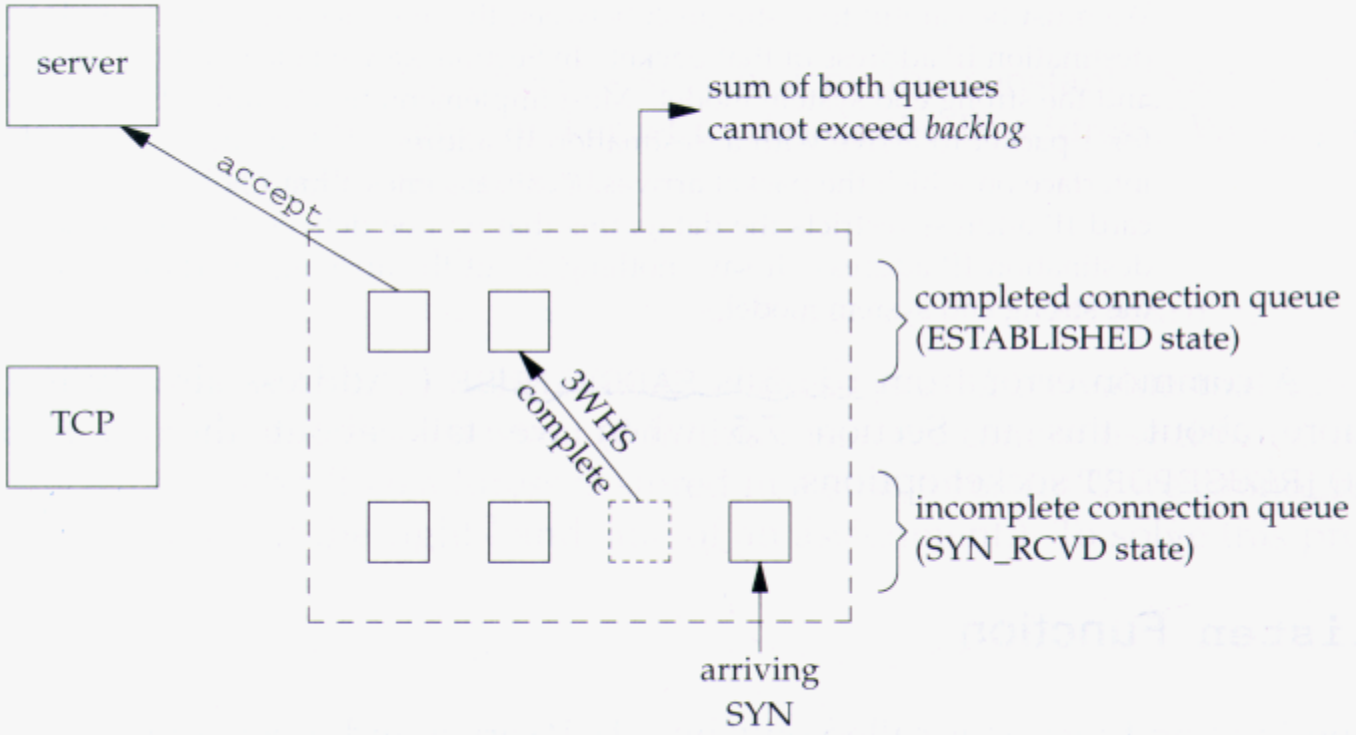


Figure 4.6 The two queues maintained by TCP for a listening socket.

listen function

- when an entry is create in the incomplete connection queue the parameters from the listen socket are copied over to the newly created connection.
- connection creation mechanism is completely automatic-server process is not involved.

listen function

figure shows packets exchanged during the connection establishment with two queues.

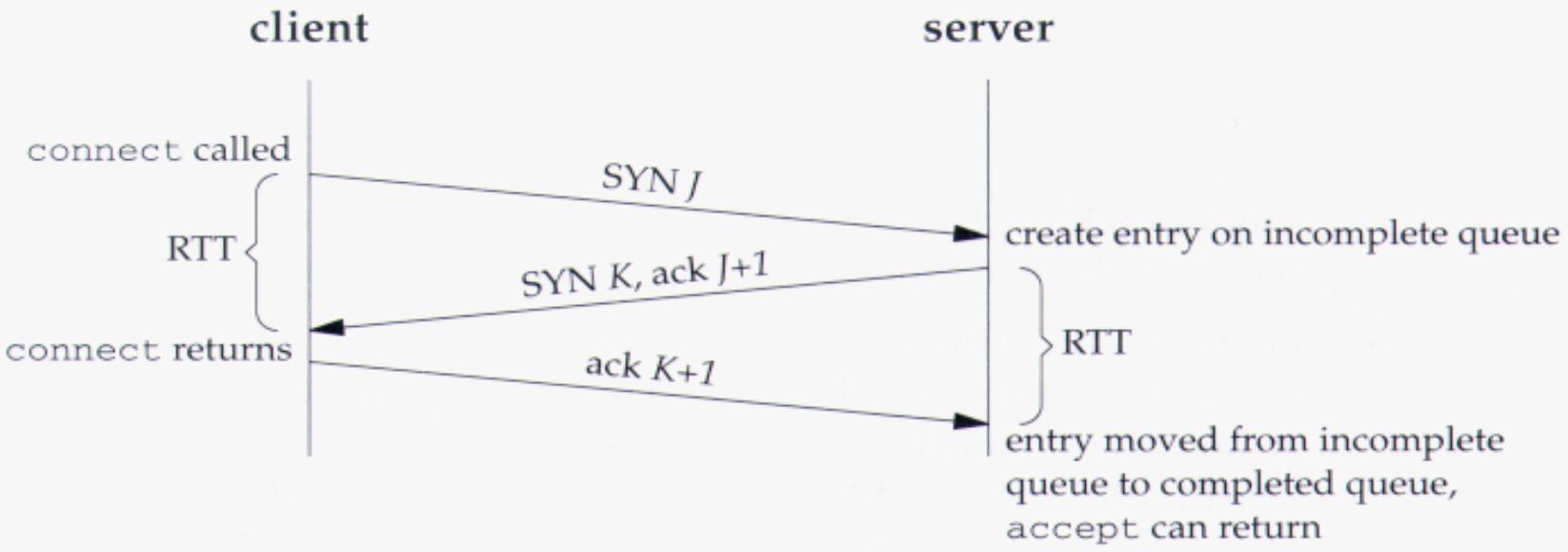


Figure 4.7 TCP three-way handshake and the two queues for a listening socket.

listen function

- when a SYN arrives from client, TCP creates a new entry on the incomplete queue and then responds with the second segment of the three-way handshake : the server's SYN with an ACK of the client's SYN.
- this entry will remain on the incomplete queue until third segment (client's ACK for the server's SYN) of the three-way handshake arrives or until the entry times out.
- if three-way handshake completes normally, the entry moves from the incomplete queue to the end of the completed queue.
- when the process calls ***accept***, the first entry on the completed queue is returned to the process, or if the queue is empty, the process is put to sleep until an entry is placed onto the completed queue.

listen function

points to consider regarding the handling of these two queues.

1. backlog- has historically specified the *maximum* value for the sum of both queues.
2. Berkeley-derived implementations add *a fudge factor* to the backlog; It is multiplied by 1.5
3. Do not specify a backlog of *0*, as different implementations interpret this differently.
4. entry in incomplete queue remains for *one RTT*, whatever the value happens between particular client and server.
5. If the *queues are full* when a SYN arrives, TCP ignores the arriving SYN; it does not send an RST.
6. Data that arrives after three-way handshake completes, before the server calls accept- should be queued by the server TCP up to the size of the connected sockets receive buffer.
7. What value should the application specify?
8. Allow Command line or an environment variable to

accept function

```
#include <sys/socket.h>
int accept(int sockfd, struct sockaddr *cliaddr,
                       socklen_t *addrlen);
```

Returns : nonnegative descriptor if OK, -1 on error

- Called by a TCP server to return the next completed connection from the front of the completed connection queue.
- Process is put to sleep- if completed connection queue is empty.
- *cliaddr* and *addrlen* are used to return the protocol address of the connected peer process(the client).

accept function

- If accept successful, returns a brand-new descriptor automatically created by the kernel.
- This new descriptor refers to the TCP connection with the client.
- *listening socket* -first argument of the accept function (server normally creates only one, exist for the life time of the server)
- *connected socket*- return value from an accept function (kernel create one connected socket for each client connection that is accepted, i.e for which TCP three-way handshake completes)

  when the server is finished serving a given client, the connected socket is closed.
- returns up to three values : an integer (brand-new descriptor or error indication), protocol address of the client process, and size of this address.
- if not interested in having the protocol address of the client returned, set both *cliaddr* and *addrlen* to null

fork and exec functions

fork function

- In unix, using the function fork() is the only way to create a new process.

```
#include <unistd.h>
  pid_t fork(void);
    Returns: 0 in child, process ID of child in parent, -1 on error
```

fork is called once but it returns twice

it returns once in the calling process(called the parent) with a return value that is the process ID of the newly created process(the child). It also returns once in the child, with a return value of 0. Hence, the return value tells the process whether it is the parent or the child.

fork function

- The reason fork return 0 in the child, instead of parent's process ID, because a child has only one parent,
- child can obtain its parent ID by calling getppid.
- parent can have any no. of children, and there is no way to obtain the PIDs of its children.
- if parent wants to keep track the PIDs of all its children, it must record return values from the fork.
- all descriptors open in the parent before the call to fork are shared with the child after fork returns.
- normally this is seen in network servers.

fork function

- two uses of fork are

1. A process can makes a copy of itself- one copy can handle one operation while the other copy does another task.
2. A process wants to execute another program. The process first calls the fork to make a copy of itself, and then one of the copies (typically the child process) calls exec to replace itself with the new program.

exec functions

The only way in which an executable program file on disk can be executed by Unix is for an existing process to call one of the six exec functions.

exec replaces the current process image with the new program file, and this new program normally starts at the main function. The process ID does not change.

process that calls exec – called the calling process

newly executed program – called the new program

exec functions

```
#include <unistd.h>
int execl(const char *pathname, const char *arg(), …/* (char *) 0*/);
int execv(const char *pathname, char *const argv[]);
int execle(const char *pathname, const char *arg());
int execve(const char *pathname, char *const argv[],
           char *const envp[]);
int execlp(const char *filename, const char *arg());
int execvp(const char *filename, char *const argv[]);
```

All six return: -1 on error, no return on success

These functions return to the caller only if an error occurs. Otherwise, control passes to the start of the new program, normally the main function.

exec functions

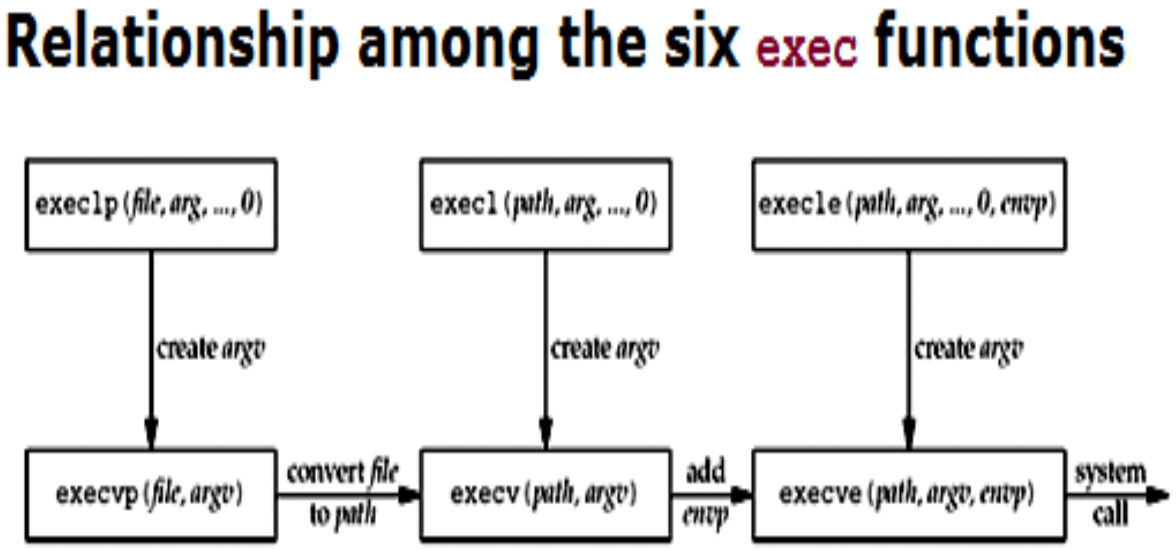
The differences in the six exec functions are:

(a) whether the program file to execute is specified by a *filename* or a *pathname*;

(b) whether the arguments to the new program are listed one by one or referenced through an array of pointers; and

(c) Whether the environment of the calling process is passed to the new program or whether a new environment is specified.

exec functions



Concurrent Servers

```
pid_t pid
int  listenfd, connfd;
listenfd = Socket(...);
//fill in sockaddr_in{} with server's well-known port
Bind(listenfd, LISTENQ);
for(;;){
      connfd = Accept(listenfd, ...);
    if( (pid = Fork()) == 0)
 {
        Close(listenfd);                    /* child closes listening
   socket */
       doit(connfd);                        //process the request
      Close();                             //done with this client
      exit(0);                             //child terminate
 }
Close(connfd);              // parent close connected socket
}
```

Figure: Outline for typical concurrent server

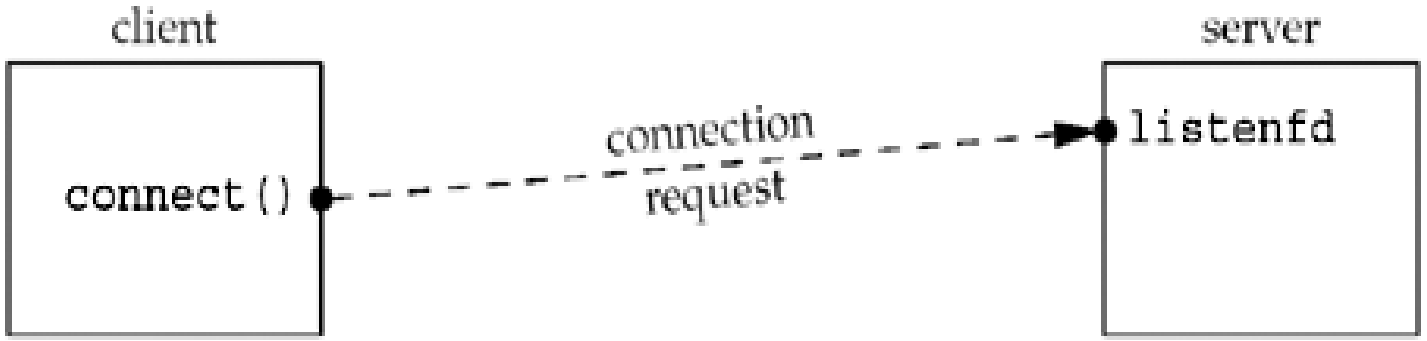


Figure 1. Status of client/server before call to accept returns.

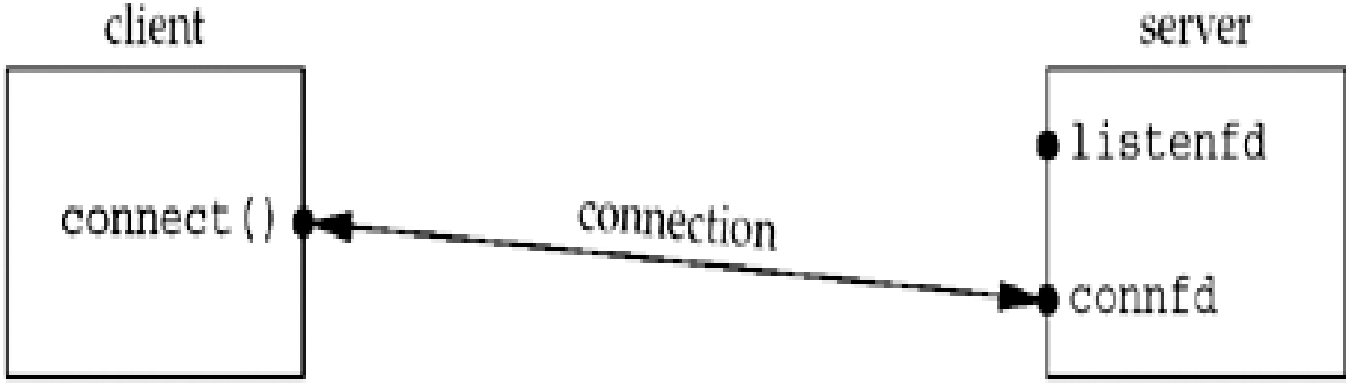


Figure 2. Status of client/server after return from accept.

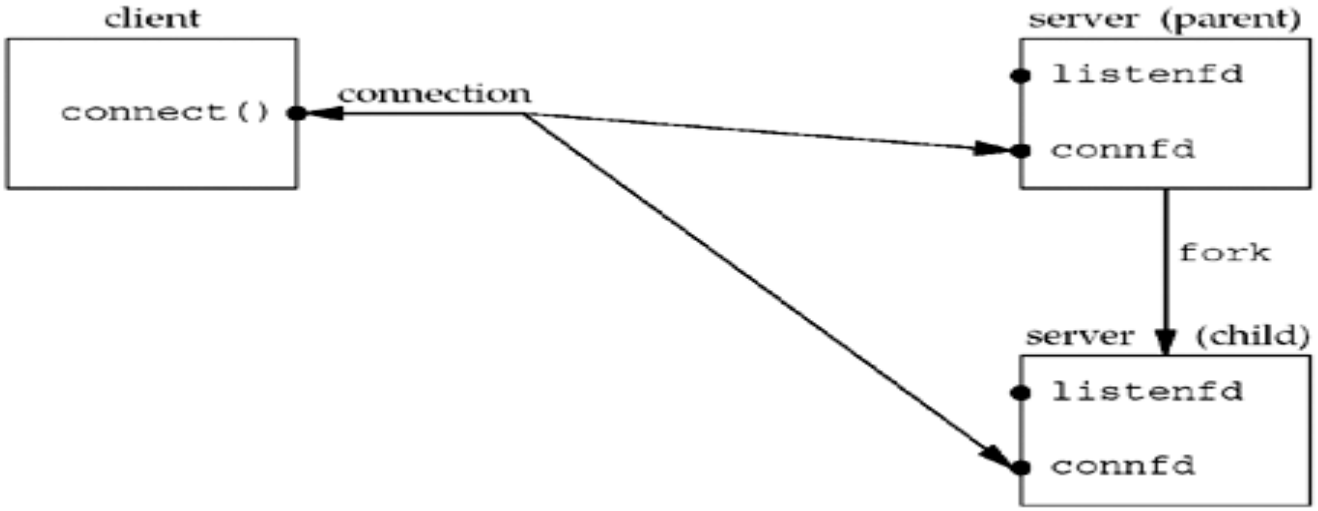


Figure 3. Status of client/server after fork returns.

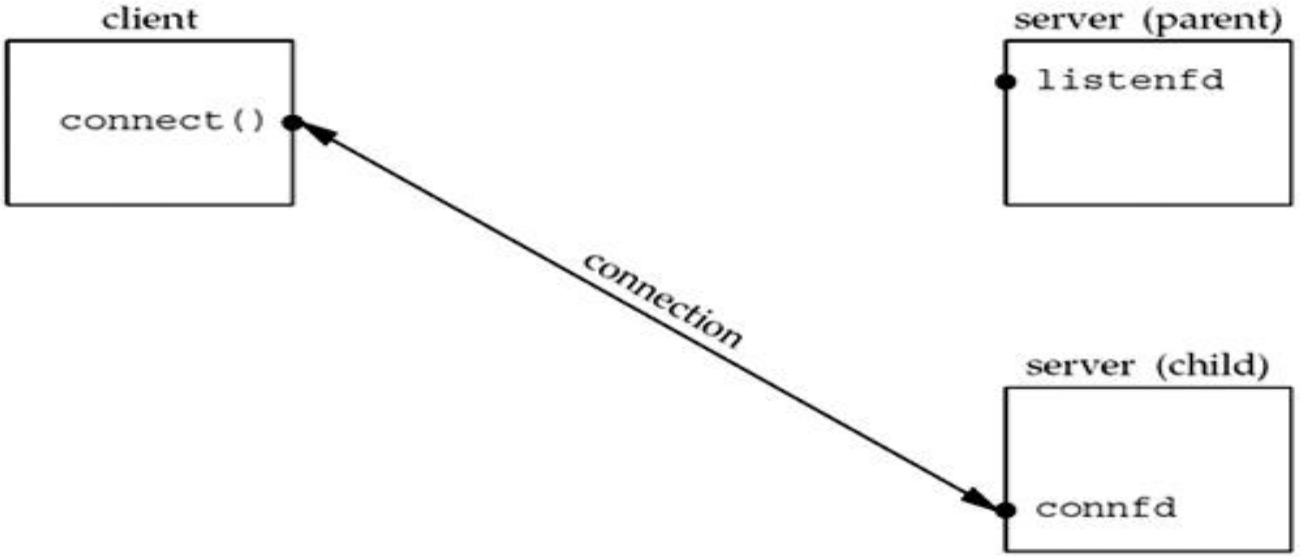


Figure 4. Status of client/server after parent and child close appropriate sockets.

Close function:-

- The normal Unix close function is also used to close a socket and terminate a TCP connection.

```
#include <unistd.h>
int close (int sockfd);
Returns: 0 if OK, -1 on error
```

- Default action of close with TCP socket is to mark the socket as closed and return to the process immediately.
- Socket descriptor is no longer usable by the process
- SO_LINGER socket option- lets us change this default action with a TCP socket.

Close function:-

Descriptor Reference Counts:-

- call to close not initiate TCP's four-packet connection termination sequence, this is the required behavior with concurrent server.
- If we want to send a FIN on TCP connection- shutdown function can be used instead of close.
- What happens in concurrent server if the parent does not call close for each connected socket returned by accept?

getsockname and getpeername Functions:-

```
#include <sys/socket.h>
int getsockname(int sockfd, struct sockaddr
  *localaddr,
                        socklen_t *addrlen);
int getpeername(int sockfd, struct sockaddr
  *peeraddr,
                        socklen_t *addrlen);
```

Both return: 0 if OK, -1 on error

getsockname: return local protocol address associated with a socket

getpeername: return foreign protocol address associated with a socket

getsockname and getpeername Functions:-

Reasons for requiring these functions

- TCP client that does not call bind, getsockname returns the local protocol address
- After calling bind with a port no. of 0, getsockname returns the local port number
- getsockname can be called to obtain the address family of a socket.
- Server binds to wildcard IP address, server can call getsockname to obtain the local IP address assigned to the connection (the socket descriptor in call is must be connected socket, not the listening socket).
- Only way to obtain the identity of client is to call getpeername

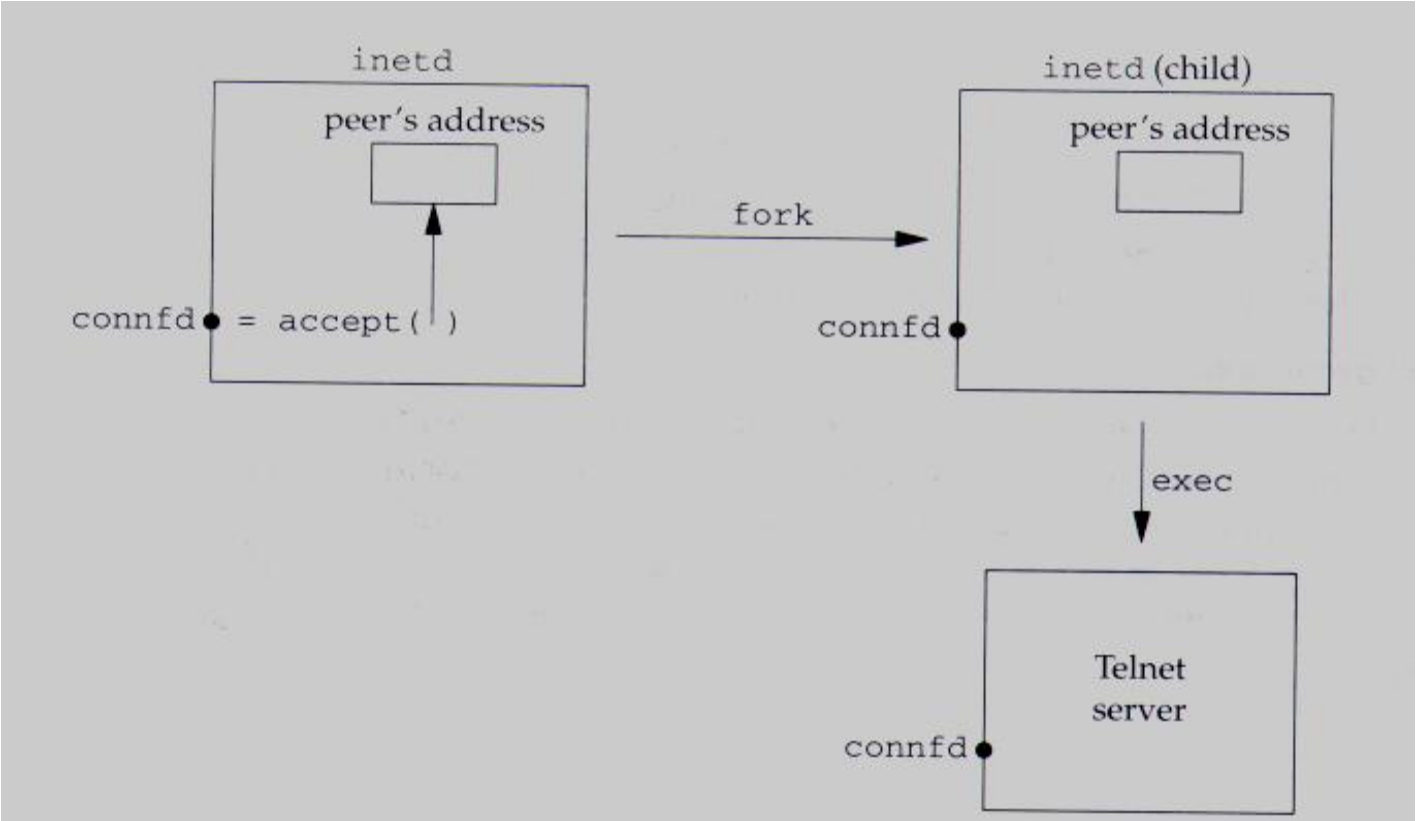


Figure 4.18 Example of `inetd` spawning a server.

TCP Client/Server Example

Contents

- Introduction
- TCP Echo Server
- TCP Echo Client
- Normal Startup and Termination
- Handling SIGCHLD Signals

Introduction

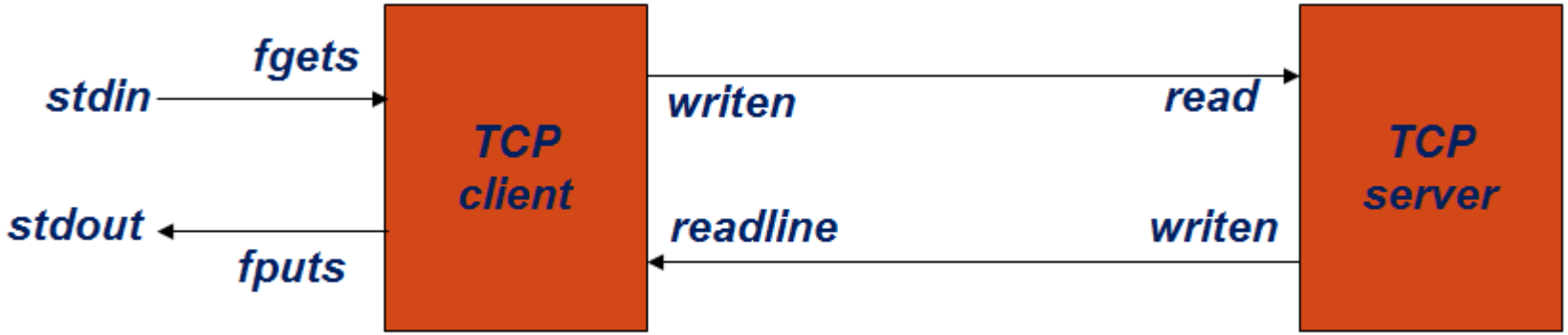


Figure: Simple echo client and server

1. **The Client reads a line of text from its standard input and writes the line to the server.**

2. **The server reads the line from its network input and echoes the line back to the client.**

3. **The client reads the echoed line and prints it on its standard output.**

TCP Echo Server: main Function

- Create socket, bind server's well-known port
- Wait for client connection to complete
- Concurrent server

Echo server main function

TCP Echo Server: str_echo Function

Performs server processing for each client : reads data from each client and echoes it back to the client.

```
#include "unp.h"
void str_echo(int sockfd)
 {
     ssize_t n;
    char buf[MAXLINE];
   again:
    while ( (n = read(sockfd, buf, MAXLINE)) > 0)
            Writen(sockfd, buf, n);
    if (n < 0 && errno == EINTR)
            goto again;
    else if (n < 0)
            err_sys("str_echo: read error");
 }
```

TCP Echo Client: main Function

- **Create socket, fill in Internet socket address structure**
- **Connect to server**

TCP Echo client main function

TCP Echo Client: str_cli function

Reads a line of text from standard input, writes it to the server, reads back the server's echo of the line, and outputs the echoed line to standard output.

Return to main function

```
#include "unp.h“
 void
str_cli(FILE *fp, int sockfd)
{
    char  sendline[MAXLINE], recvline[MAXLINE];
    while (Fgets(sendline, MAXLINE, fp) != NULL) {
          Writen(sockfd, sendline, strlen (sendline));

    if (Readline(sockfd, recvline, MAXLINE) == 0)
          err_quit("str_cli: server terminated prematurely");
     Fputs(recvline, stdout);

    }
}
```

Normal Startup

- how the client and server start, how they end, and most importantly, what happens when something goes wrong: the client host crashes, the client process crashes, network connectivity is lost and so on.

first we start server in background process on the linux host

- linux % **tcpserv01 &**

  [1] 17870

Calling netstat program to verify the state of the server's listening socket.

- linxux % **netstat -a**

Active Internet connections (servers and established)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	* : 9877	* : *	LISTEN

Next start the client on the same host, specifying the server's

Normal Startup

- linux % tcpcli01 127.0.0.1
- the client calls socket and connect, it causes the three-way handshake to taken place.
- When the three-way handshake completes, connect returns in the client and accept returns in the server.

- The connection is established. The following steps then take place:

1. The client calls str_cli, which will block in the call to fgets, because we have not typed a line of input yet.

Normal Startup

2. When accept returns in the server, it calls fork and the child calls str_echo. This function calls readline, which calls read, which blocks while waiting for a line to be sent from the client.

3. The server parent, on the other hand, calls accept again, and blocks while waiting for the next client connection.

Now three processes, and all three are asleep (blocked): client, server parent, and server child.

Normal Startup

Using netstat verirfying the state of the sockets

Linux % netstat -a

Active Internet connections (servers and established)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	local host:9877	localhost:42758	ESTABLISHED
tcp	0	0	local host:42758	localhost:9877	ESTABLISHED
tcp	0	0	*:9877	*:*	LISTEN

Using ps command to checking the status and relationship of these processes

Linux ps -t pts/6 -o pid, ppid, tty, stat, args, wchan

PID	PPID	TT	STAT	COMMAND	WCHAN
22038	22036	pts/6	S	–bash	wait4
17870	22038	pts/6	S	./tcpserv01	wait_for_connect
19315	17870	pts/6	S	./tcpserv01	tcp_data_wait
19314	22038	pts/6	S	./tcpcli01127.0.0.1	read_chan

S- Means Sleeping

WCHAN – specifies the condition when process is asleep(blocked)

Normal Termination

At this point, the connection is established and whatever we type to the client is echoed back.

Linux % tcpcli01 127.0.0.1 // we showed this line earlier
hello, world // we now type this
hello, world // and the line is echoed
good bye
good bye
^D //Control-D is our terminal EOF character

We type in two lines, each one is echoed, and then we type our terminal EOF character (Control-D), which terminates the client.

Steps involved in the normal termination of our client and server:

TCP connection termination diagram

1. **When we type our EOF character, fgets returns a null pointer and the function str_cli returns.**

2. **When str_cli returns to the client main function ,the latter terminates by calling exit.**

3. **Part of process termination is the closing of all open descriptors, so the client socket is closed by the kernel. This sends a FIN to the server, to which the server TCP responds with an ACK. This is the first half of the TCP connection termination sequence.**

Steps involved in the normal termination of our client and server:

4. When the server TCP receives the FIN, the server child is blocked in a call to readline ,and readline then returns 0. This causes the str_echo function to return to the server child main.

5. The server child terminates by calling exit.

6. All open descriptors in the server child are closed. The closing of the connected socket by the child causes the final two segments of the TCP connection termination to take place: a FIN from the server to the client, and an ACK from the client. At this point, the connection is completely terminated.

7. Finally, the SIGCHLD signal is sent to the parent when the server child terminates.

Steps involved in the normal termination of our client and server:

linux % **ps -t pts/6 -o pid, ppid, tty, stat, args, wchan**

PID	PPID	TT	STAT	COMMAND	WCHAN
22038	22036	pts/6	S	- bash	read_chan
17870	22038	pts/6	S	./tcpserv01	wait_for_connect
19315	17870	pts/6	Z	tcpserv01	<defu do_exit

The STAT of the child is now Z (for zombie).

POSIX Signal Handling

Signal- is a notification to a process that an event occurred.

- Sometimes called *software interrupts*
- Can occur asynchronously.
- Can be sent in two ways
  - By one process to another process (or to itself)
  - By the kernel to a process

Ex:- SIGCHLD

- Has associated *disposition*, also called *action* associated with the signal.
- sigaction function –sets the disposition of a signal

POSIX Signal Handling

Three choices for the disposition

1. Providing a function- called signal handler, this action is called catching a signal.

   SIGKILL and SIGSTOP cannot be caught.

   prototype void handler(int signo);

   SIGIO, SIGPOLL, and SIGURG-require additional actions on part of process to catch the signal.

2. Ignoring the signal – SIG_IGN

   SIGKILL and SIGSTOP cannot be ignored.

3. Setting default disposition for a signal- SIG_DFL

   default- terminate a process on receipt of a signal, copying core image of

   the process in the current working directory.

   SIGCHLD and SIGURG- default is ignored.

Signal function:-

- POSIX way to establish the disposition of a signal is to call sigaction function.
- It is complicated- one argument to this function is a structure.
- Easier way is to call the *signal* function- the first argument is signal name, second argument is either a pointer to a function or one of the constants SIG_IGN or SIG_DFL.
- It just calls the POSIX sigaction function
- signal is an historical function that predates POSIX
- This provides simple interface with desired POSIX semantics.
- code for signal function

POSIX Signal Semantics:-

- Once a signal handler is installed, it remains installed.
- While a signal handler is executing, the signal being delivered is blocked.
- If a signal is generated one or more times while it is blocked, it is normally delivered only one time after the signal is unblocked.
- It is possible to selectively block and unblock a set of signals using the ***sigprocmask*** function.

Termination of Server Process

What happens to the client?

The following steps take place:

1. We start the server and client and verify that all is okay.
2. Find the process ID of the server child and kill it, all open descriptors in the child are closed. This causes a FIN to be sent to the client, and the client TCP responds with an ACK.

   This is the first half of the TCP connection termination.
3. The SIGCHLD signal is sent to the server parent and handled correctly.
4. Nothing happens at the client. The client TCP receives the FIN from the server TCP and responds with an ACK, but the problem is that the client

Termination of Server Process

5. Running netstat at this point shows the state of the sockets.

```
linux % netstat -a | grep 9877
tcp    0     0    *: 9877              *:*                                  LISTEN
tcp     0     0    localhost:9877          localhost:43604
FIN_WAIT2
tcp             1     0    localhost:43604          localhost:9877
CLOSE_WAIT
```

Termination of Server Process

6. Type a line of input to the client.

Here is what happens at the client starting from Step 1:

linux % tcpcli01 127.0.0.1 start client

Hello the first line that we type

Hello is echoed correctly

here we kill the server child on the server host

another line we then type a second line to the client

str_cli : server terminated prematurely

When the server TCP receives the data from the client, it responds with an RST since the process that had that

Termination of Server Process

7. The client process will not see the RST because it calls readline immediately after the call to writen and readline returns 0 (EOF) immediately because of the FIN that was received in Step 2. Our client is not expecting to receive an EOF at this point so it quits with the error message "server terminated prematurely.“

8. When the client terminates (by calling err_quit), all its open descriptors are closed.

The above described also depends on the timing of the example.

If the RST arrives first, the result is an ECONNRESET ("Connection reset by peer") error return from readline.

Crashing of Server Host

(Sever host unreachable)

What happens when the server host crashes?

The following steps take place:

1. When the server host crashes, nothing is sent out on the existing network connections.(assume that server host crashes and not shutdown by the operator)
2. We type a line of input to the client, it is written by writen , and is sent by the client TCP as a data segment. The client then blocks in the call to readline, waiting for the echoed reply.

Crashing of Server Host

(Sever host unreachable)

3. The client TCP continually retransmitting the data segment, trying to receive an ACK from the server. When the client TCP finally gives up, an error is returned to the client process. Since the client is blocked in the call to readline, it returns an error. Assuming the server host crashed and there were no responses at all to the client's data segments, the error is ETIMEDOUT. But if some intermediate router determined that the server host was unreachable and responded with an ICMP "destination unreachable' message, the error is either EHOSTUNREACH or ENETUNREACH.

- The scenario that just discussed detects that the server host has crashed only when we send data to that host.
- Other techniques are there with SO_KEEPALIVE socket option.

Crashing and Rebooting of Server Host

The following steps take place:

1. We start the server and then the client. Type a line to verify that the connection is established.

2. The server host crashes and reboots.

3. Type a line of input to the client, which is sent as a TCP data segment to the server host.

Crashing and Rebooting of Server Host

4. When the server host reboots after crashing, its TCP loses all information about connections that existed before the crash. Therefore, the server TCP responds to the received data segment from the client with an RST.

5. Client is blocked in the call to readline when the RST is received, causing readline to return the error ECONNRESET.

Shutdown of Server Host

What happens if the server host is shut down by an operator while our server process is running on that host?

- init process normally sends the SIGTERM signal to all processes.
- It waits some fixed amount of time (often between 5 and 20 seconds), and then sends the SIGKILL signal to any processes still running.
- This gives all running processes a short amount of time to clean up and terminate.
- If we do not catch SIGTERM and terminate, our server will be terminated by the SIGKILL signal.
- When the process terminates, all open descriptors are closed, and we then follow the same sequence of steps in case of ***termination of server process*** .
- Client can use select or poll functions to detect the termination of the server process as soon as it occurs.